

INFO-F-302, Cours d'Informatique Fondamentale

Emmanuel Filiot et Jean-François Raskin

Département d'Informatique

Faculté des Sciences

Université Libre de Bruxelles

Année académique 2026-2027

Le Problème SAT

3 Rappels

Satisfaisabilité

Soit P un ensemble de propositions. Une formule ϕ de la logique propositionnelle sur P est *satisfaisable* si

3 Rappels

Satisfaisabilité

Soit P un ensemble de propositions. Une formule ϕ de la logique propositionnelle sur P est *satisfaisable* si il existe une interprétation $V : X \rightarrow \{1, 0\}$ telle que $V \models \phi$.

3 Rappels

Satisfaisabilité

Soit P un ensemble de propositions. Une formule ϕ de la logique propositionnelle sur P est *satisfaisable* si il existe une interprétation $V : X \rightarrow \{1, 0\}$ telle que $V \models \phi$.

Dans ce chapitre, nous allons utiliser des solveurs SAT, qui permettent de tester la satisfaisabilité de formules représentées comme des **ensembles de clauses**.

4 Littéraux et Clauses

Définition

- ▶ Un littéral est une variable x ou sa négation $\neg x$
- ▶ Une clause est une disjonction de littéraux $\ell_1 \vee \dots \vee \ell_n$. Elle est satisfaite par une valuation V s'il existe i tel que $V(\ell_i) = 1$. Par extension, on appelle $\perp$ la *clause vide*, qui est insatisfaisable.

Donc, p et $\neg p$ sont des littéraux. $x \vee \neg y \vee r$ est une clause, mais pas $x \wedge y$.

4 Littéraux et Clauses

Définition

- ▶ Un littéral est une variable x ou sa négation $\neg x$
- ▶ Une clause est une disjonction de littéraux $\ell_1 \vee \dots \vee \ell_n$. Elle est satisfaite par une valuation V s'il existe i tel que $V(\ell_i) = 1$. Par extension, on appelle $\perp$ la *clause vide*, qui est insatisfaisable.

Donc, p et $\neg p$ sont des littéraux. $x \vee \neg y \vee r$ est une clause, mais pas $x \wedge y$.

Satisfaction d'ensemble de clauses

Un ensemble de clauses $A = \{C_1, \dots, C_n\}$ est satisfait par une valuation V , noté $V \models A$, si pour tout i , $V \models C_i$. En particulier, tout valuation satisfait l'ensemble vide $A = \emptyset$.

Lien avec les formes normales

Une formule est en *forme normale conjonctive* (FNC) ssi c'est une conjonction de disjonctions de littéraux, de la forme : de la forme :

$$\bigwedge_i (\bigvee_j (\neg) x_{i,j})$$

Une formule est en *forme normale disjonctive* (FND) ssi c'est une disjonction de conjonctions de littéraux, de la forme : de la forme :

$$\bigvee_i (\bigwedge_j (\neg) x_{i,j})$$

Lien avec les formes normales

Une formule est en *forme normale conjonctive* (FNC) ssi c'est une conjonction de disjonctions de littéraux, de la forme : de la forme :

$$\bigwedge_i (\bigvee_j (\neg)x_{i,j})$$

Une formule est en *forme normale disjonctive* (FND) ssi c'est une disjonction de conjonctions de littéraux, de la forme : de la forme :

$$\bigvee_i (\bigwedge_j (\neg)x_{i,j})$$

Lemme

Tout ensemble non-vide de clauses $A = \{C_1, \dots, C_n\}$ est équivalent à la formule en FNC $\phi_A = \bigwedge_{i=1}^n C_i$, au sens où pour toute valuation, $V \models A$ ssi $V \models \phi_A$.

⇒ Pour mettre une formule sous forme de clauses, il suffit de la mettre en FNC !

6 Mise sous FNC et FND

On utilise les transformations successives suivantes pour obtenir les formes normales :

1. élimination des connecteurs $\rightarrow$ et $\leftrightarrow$ grâce aux équivalences suivantes :

$$\begin{aligned}(\phi \rightarrow \psi) &\equiv (\neg\phi \vee \psi) \\ (\phi \leftrightarrow \psi) &\equiv (\neg\phi \vee \psi) \wedge (\phi \vee \neg\psi)\end{aligned}$$

2. entrer les négations le plus à l'intérieur possible :

$$\begin{aligned}\neg(\phi \wedge \psi) &\equiv (\neg\phi \vee \neg\psi) & \neg\neg\phi &\equiv \phi \\ \neg(\phi \vee \psi) &\equiv (\neg\phi \wedge \neg\psi)\end{aligned}$$

3. utilisation des distributivité de $\vee$ et $\wedge$:

$$\begin{aligned}(\phi \vee (\psi \wedge \chi)) &\equiv (\phi \vee \psi) \wedge (\phi \vee \chi) \text{ (pour mise sous FNC)} \\ (\phi \wedge (\psi \vee \chi)) &\equiv (\phi \wedge \psi) \vee (\phi \wedge \chi) \text{ (pour mise sous FND)}\end{aligned}$$

Mise en FNC de $\neg(x \leftrightarrow (y \rightarrow r))$

Mise en FNC de $\neg(x \leftrightarrow (y \rightarrow r))$

1. on retire les équivalences et implications :

$$\neg((\neg x \vee \neg y \vee r) \wedge (\neg(\neg y \vee r) \vee x))$$

2. on pousse les négations à l'intérieur :

$$(x \wedge y \wedge \neg r) \vee ((\neg y \vee r) \wedge \neg x)$$

3. on distribue :

$$((x \wedge y \wedge \neg r) \vee (\neg y \vee r)) \wedge ((x \wedge y \wedge \neg r) \vee \neg x)$$

4. et encore :

$$(x \vee \neg y \vee r) \wedge (y \vee \neg y \vee r) \wedge (\neg r \vee \neg y \vee r) \wedge (x \vee \neg x) \wedge (y \vee \neg x) \wedge (\neg r \vee \neg x)$$

5. on retire les formules équivalentes à $\top$:

$$(x \vee \neg y \vee r) \wedge (y \vee \neg x) \wedge (\neg r \vee \neg x)$$

9 Exercice

Mettre les formules suivantes sous FNC :

1. $\neg(x \rightarrow q)$

2. $(x \wedge q) \vee r$

3. $x \wedge (y \vee (r \wedge t))$

4. $\neg(x \wedge ((r \vee q) \wedge s))$

5. $\neg(x \wedge (r \leftrightarrow q))$

6. $x \leftrightarrow (y \vee r)$

7. $(x_1 \wedge q_1) \vee (x_2 \wedge q_2) \vee (x_3 \wedge q_3)$

8. généraliser aux formules de la forme

$(x_1 \wedge q_1) \vee (x_2 \wedge q_2) \vee \cdots \vee (x_n \wedge q_n)$ pour tout $n \geq 0$

10 Problème SAT

Définition

ENTREE Un ensemble de clauses S .

SORTIE Est-ce que S est satisfaisable ?

Dans le cas où S est satisfaisable, on aimerait que l'algorithme nous retourne une valuation V qui satisfait S .

11 Solveurs SAT

- ▶ Un solveur SAT est un programme qui *décide* le problème SAT, i.e. qu'étant donné un ensemble de clauses S , il retourne le Booléen `true` si et seulement si S est satisfaisable.
- ▶ De plus, si S satisfaisable, une interprétation qui le satisfait est retournée.

12 Motivations

- ▶ Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- ▶ Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).

12 Motivations

- ▶ Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- ▶ Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).
- ▶ Les problèmes de la classe NP (<http://www.nada.kth.se/~viggo/problemlist/>) se réduisent (i.e. se "transforment") tous au problème SAT en temps polynomial.

12 Motivations

- ▶ Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- ▶ Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).
- ▶ Les problèmes de la classe NP (<http://www.nada.kth.se/~viggo/problemlist/>) se réduisent (i.e. se "transforment") tous au problème SAT en temps polynomial.
- ▶ **Motivation principale** : Plutôt que d'avoir un algorithme pour chaque problème, écrire un bon algorithme pour SAT et transformer automatiquement chaque instance du problème de départ en une instance de SAT équivalente.

12 Motivations

- ▶ Beaucoup de problèmes s'expriment naturellement par des formules en FNC, et donc sous formes d'ensemble de clauses
- ▶ Par exemple, un problème de planification où l'on a beaucoup de contraintes (une conjonction) et chaque contrainte représente plusieurs alternatives possibles (disjonction).
- ▶ Les problèmes de la classe NP (<http://www.nada.kth.se/~viggo/problemelist/>) se réduisent (i.e. se "transforment") tous au problème SAT en temps polynomial.
- ▶ **Motivation principale** : Plutôt que d'avoir un algorithme pour chaque problème, écrire un bon algorithme pour SAT et transformer automatiquement chaque instance du problème de départ en une instance de SAT équivalente.
- ▶ Si la transformation (aussi appelée réduction) se fait dans un temps raisonnable (polynomiale), alors l'utilisation d'un solveur SAT donne parfois d'aussi bons résultats qu'un algorithme dédié au problème de départ.

13 Histoire des Solveurs SAT

- ▶ **Davis & Putnam, 1960 ; Davis, Longemann & Loveland, 1962**
Algorithme **DPLL**.
- ▶ **Tseitin, 1966** DPLL a une complexité en temps exponentiel au pire cas
- ▶ **Cook 1971** SAT est NP-complet
- ▶ Seconde Génération de solveurs SAT basés sur DPLL : Posit [1995], SATO [1997], GRASP [1999].
- ▶ Compétition SAT depuis 2002 :
<http://www.satcompetition.org/>
- ▶ Solveurs SAT très efficaces : Chaff [2001], BerkMin [2002], zChaff [2004], MiniSAT [2005] ... Regroupent différentes techniques d'optimisation telles que l'apprentissage, des structures de données efficaces pour stocker les clauses, des heuristiques etc ...

14 Notations pour les grandes disjonctions et conjonctions

Définition

Disjonction et conjonction

- ▶ On appellera *disjonction* une formule de la forme $\phi_1 \vee \phi_2 \vee \cdots \vee \phi_n$, où $n \geq 1$. Attention lorsque $n = 1$, la disjonction est réduite à la seule formule ϕ_1 . On utilisera l'abréviation

$$\bigvee_{i=1}^n \phi_i$$

pour noter cette disjonction.

- ▶ On appellera *conjonction* une formule de la forme $\phi_1 \wedge \phi_2 \wedge \cdots \wedge \phi_n$, où $n \geq 1$. Attention lorsque $n = 1$, la conjonction est réduite à la seule formule ϕ_1 . On utilisera l'abréviation

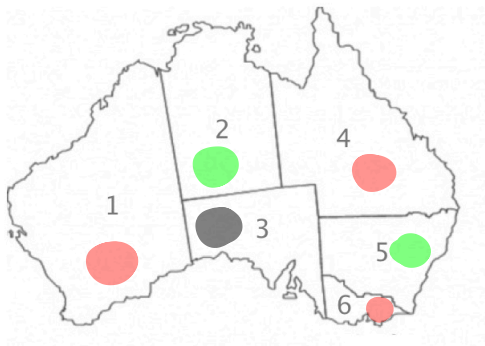
$$\bigwedge_{i=1}^n \phi_i$$

15 Modélisation

Nous allons maintenant nous intéresser à la modélisation de plusieurs problèmes en logique propositionnelle :

- ▶ le coloriage de cartes
- ▶ le problème des 8 reines
- ▶ le jeu Sudoku

16 Coloriage de Cartes



- ▶ colorier chaque région de telle sorte que deux régions ayant une frontière commune n'aient jamais la même couleur
- ▶ but : minimiser le nombre de couleurs

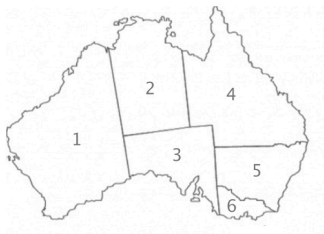
17 Coloriage de Cartes - Modélisation



Questions

- combien faut-il de couleur minimum pour la carte du dessus ?

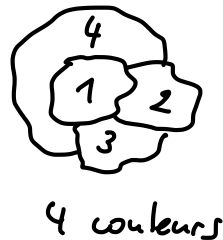
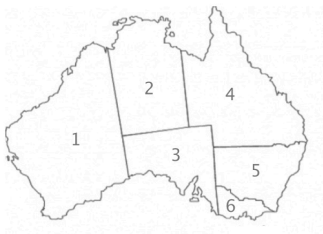
17 Coloriage de Cartes - Modélisation



Questions

- combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs

17 Coloriage de Cartes - Modélisation

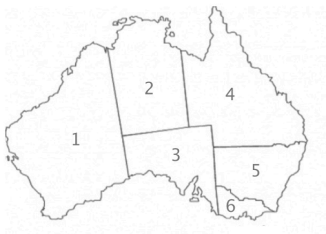


4 couleurs

Questions

- ▶ combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs
- ▶ pourriez-vous donner une carte où il faut au moins 4 couleurs ?

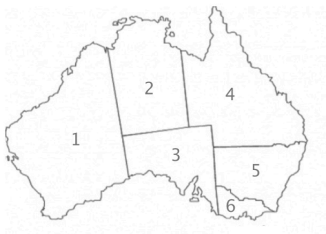
17 Coloriage de Cartes - Modélisation



Questions

- ▶ combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs
- ▶ pourriez-vous donner une carte où il faut au moins 4 couleurs ?
- ▶ et 5 couleurs ?

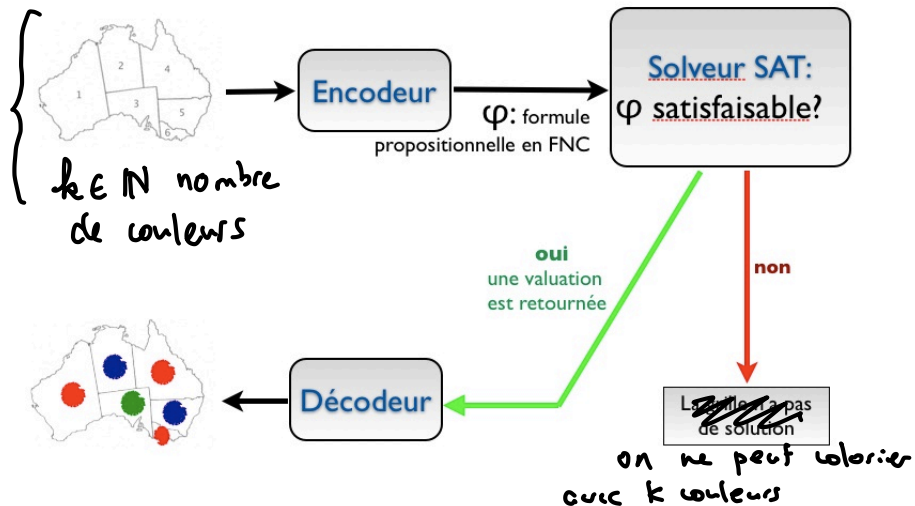
17 Coloriage de Cartes - Modélisation



Questions

- ▶ combien faut-il de couleur minimum pour la carte du dessus ? 3 couleurs
- ▶ pourriez-vous donner une carte où il faut au moins 4 couleurs ?
- ▶ et 5 couleurs ?
- ▶ **Remarque :** lorsque les régions ne sont pas discontinues (i.e. qu'une région est nécessairement en un seul morceau), alors 4 couleurs suffisent toujours (*Théorème de 4 couleurs*)

Modélisation : Rappel



Coloriage de Cartes - Modélisation

Choix des Propositions

- ▶ fixons un nombre de couleurs (3) et exprimons l'existence d'un coloriage avec 3 couleurs en logique propositionnelle.
- ▶ la première étape
- ▶ ici, l'ensemble des régions est $\{1, 2, 3, 4, 5, 6\}$ et l'ensemble des couleurs est $\{R, V, B\}$. Pour chaque région r et chaque couleur c , nous introduisons la proposition “la région r est coloriée avec la couleur c ”, notée $x_{r,c}$. On a donc l'ensemble des variables suivant :

$$X = \{x_{r,c} \mid r \in \{1, \dots, 6\}, c \in \{R, V, B\}\}$$

$$x_{1,R} = 1$$

$$x_{1,B} = 1$$

$$x_{1,V} = 1$$

$$x_{2,R} = 1$$

$$x_{2,B} = 1$$

$$x_{2,V} = 1$$

$\vdots$

On va vouloir

exclure ce type de

solution en exprimant

qu'il existe au plus

une couleur par région

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- ▶ les régions voisines doivent avoir des couleurs différentes, ici les régions voisines sont données par l'ensemble des paires
 $\text{Voisines} = \{(1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (3, 5), (5, 6), (3, 6)\}.$

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- ▶ les régions voisines doivent avoir des couleurs différentes, ici les régions voisines sont données par l'ensemble des paires
 $\text{Voisines} = \{(1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (3, 5), (5, 6), (3, 6)\}.$
- ▶ comment exprimer que 1 et 2 ont des couleurs différentes ?

$\neg (X_{1,R} \wedge X_{2,R})$ "1 et 2 n'ont pas la couleur rouge"

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- ▶ les régions voisines doivent avoir des couleurs différentes, ici les régions voisines sont données par l'ensemble des paires
 $\text{Voisines} = \{(1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (3, 5), (5, 6), (3, 6)\}.$
- ▶ comment exprimer que 1 et 2 ont des couleurs différentes ?

$$\begin{aligned}
 & \neg((x_{1,V} \wedge x_{2,V}) \vee (x_{1,B} \wedge x_{2,B}) \vee (x_{1,R} \wedge x_{2,R})) \\
 \equiv & (\neg x_{1,V} \vee \neg x_{2,V}) \wedge (\neg x_{1,B} \vee \neg x_{2,B}) \wedge (\neg x_{1,R} \vee \neg x_{2,R}) \\
 \equiv & \bigwedge_{c \in \{R,V,B\}} \neg x_{1,c} \vee \neg x_{2,c}
 \end{aligned}$$

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- Il faut exprimer la contrainte pour toutes les paires voisines de $Voisines = \{(1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (3, 5), (5, 6), (3, 6)\}$:

$$\begin{aligned}
 & (\neg x_{1,V} \vee \neg x_{2,V}) \wedge (\neg x_{1,B} \vee \neg x_{2,B}) \wedge (\neg x_{1,R} \vee \neg x_{2,R}) \\
 \wedge & (\neg x_{1,V} \vee \neg x_{3,V}) \wedge (\neg x_{1,B} \vee \neg x_{3,B}) \wedge (\neg x_{1,R} \vee \neg x_{3,R}) \\
 \wedge & (\neg x_{2,V} \vee \neg x_{4,V}) \wedge (\neg x_{2,B} \vee \neg x_{4,B}) \wedge (\neg x_{2,R} \vee \neg x_{4,R}) \\
 \wedge & (\neg x_{3,V} \vee \neg x_{4,V}) \wedge (\neg x_{3,B} \vee \neg x_{4,B}) \wedge (\neg x_{3,R} \vee \neg x_{4,R}) \\
 \wedge & (\neg x_{4,V} \vee \neg x_{5,V}) \wedge (\neg x_{4,B} \vee \neg x_{5,B}) \wedge (\neg x_{4,R} \vee \neg x_{5,R}) \\
 \wedge & (\neg x_{3,V} \vee \neg x_{5,V}) \wedge (\neg x_{3,B} \vee \neg x_{5,B}) \wedge (\neg x_{3,R} \vee \neg x_{5,R}) \\
 \wedge & (\neg x_{5,V} \vee \neg x_{6,V}) \wedge (\neg x_{5,B} \vee \neg x_{6,B}) \wedge (\neg x_{5,R} \vee \neg x_{6,R}) \\
 \wedge & (\neg x_{3,V} \vee \neg x_{6,V}) \wedge (\neg x_{3,B} \vee \neg x_{6,B}) \wedge (\neg x_{3,R} \vee \neg x_{6,R})
 \end{aligned}$$

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- Il faut exprimer la contrainte pour toutes les paires voisines de
Voisines = $\{(1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (3, 5), (5, 6), (3, 6)\}$:

$$\begin{aligned}
 & (\bigwedge_{c \in \{R, V, B\}} \neg x_{1,c} \vee \neg x_{2,c}) \\
 \wedge & (\bigwedge_{c \in \{R, V, B\}} \neg x_{1,c} \vee \neg x_{3,c}) \\
 \wedge & (\bigwedge_{c \in \{R, V, B\}} \neg x_{2,c} \vee \neg x_{4,c}) \\
 \wedge & (\bigwedge_{c \in \{R, V, B\}} \neg x_{3,c} \vee \neg x_{4,c}) \\
 \wedge & (\bigwedge_{c \in \{R, V, B\}} \neg x_{4,c} \vee \neg x_{5,c}) \\
 \wedge & (\bigwedge_{c \in \{R, V, B\}} \neg x_{3,c} \vee \neg x_{5,c}) \\
 \wedge & (\bigwedge_{c \in \{R, V, B\}} \neg x_{5,c} \vee \neg x_{6,c}) \\
 \wedge & (\bigwedge_{c \in \{R, V, B\}} \neg x_{3,c} \vee \neg x_{6,c})
 \end{aligned}$$

Coloriage de Cartes - Modélisation



Expression de la contrainte de bon coloriage

- Il faut exprimer la contrainte pour toutes les paires voisines de $Voisines = \{(1, 2), (1, 3), (2, 4), (3, 4), (4, 5), (3, 5), (5, 6), (3, 6)\}$:

$$\phi = \bigwedge_{(i,j) \in Voisines} \bigwedge_{c \in \{R,V,B\}} \neg x_{i,c} \vee \neg x_{j,c}$$

$\phi =$

$$\bigwedge_{(i,j) \in \text{Voisines}} \bigwedge_{c \in \{R,V,B\}}$$

$$\neg X_{i,c} \vee \neg X_{j,c}$$

Pour toute paire (i,j) de régions voisines,

Pour toute couleur c dans $\{R, V, B\}$,

i	n'a	pas	la	couleur	c	ou
j	"	"	"	"	"	

" la région 1 possède au moins une couleur "

$$X_{1,R} \vee X_{1,B} \vee X_{1,V}$$

$\leadsto$ le faire pour toutes les régions.

Coloriage de Cartes - Modélisation

- ▶ La formule précédente est satisfaisable même si on met toutes les variables à 0
- ▶ Autrement dit, si on ne colorie rien, alors pas de risque de colorier deux régions voisines avec la même couleur !
- ▶ il manque la contrainte qu'il faut que chaque région soit coloriée

Expression de la contrainte d'existence d'une couleur pour chaque région

- ▶ On exprime que pour toute région, il existe une couleur qui la colorie :

$$\phi = \bigwedge_{r \in \{1, \dots, 6\}} \left(\bigvee_{c \in \{R, V, B\}} x_{r,c} \right)$$

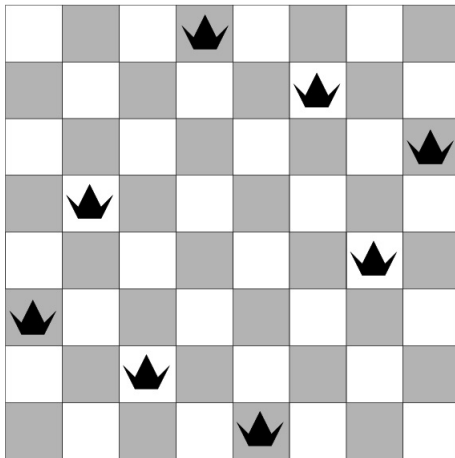
25 Coloriage de graphes

- ▶ On peut généraliser le problème précédent des cartes aux graphes.
- ▶ Un graphe est coloriable avec k couleurs s'il existe une fonction qui associe à chaque sommet une unique couleur, telle que les sommets adjacents n'ont pas la même couleur
- ▶ Soit un graphe (fini) $G = (S, A)$ avec S l'ensemble des sommets et $A \subseteq V \times V$ l'ensemble d'arêtes, et soit k le nombre de couleurs.
- ▶ Pour tout $s \in S$ et $c \in \{1, \dots, k\}$, on pose $x_{s,c}$ la proposition " s est coloriée avec c ".
- ▶ La formule

$$\left(\bigwedge_{(u,u') \in A} \bigwedge_{1 \leq c < c' \leq k} \neg x_{u,c} \vee \neg x_{u',c'} \right) \wedge \bigwedge_{u \in S} \left(\bigvee_{1 \leq c \leq k} x_{u,c} \right)$$

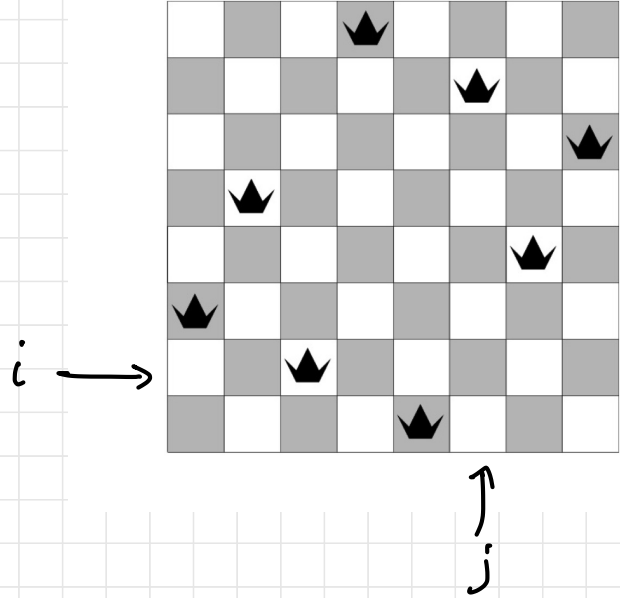
est satisfaisable ssi le graphe G est coloriable avec k couleurs au plus.

26 Problème des 8 reines (Au Tableau)



CHOIX DES VARIABLES

X_{ij} : "il y a exactement
une reine à la
ligne i et
colonne j "



CONTRAINTES

"au moins une reine par ligne"

Pour toute ligne $i \in \{1, \dots, 8\}$,

il existe une colonne $j \in \{1, \dots, 8\}$,

(i, j) contient une reine.

$$\varphi_1 = \left(\bigvee_{j \in \{1, \dots, 8\}} x_{i,j} \right)$$

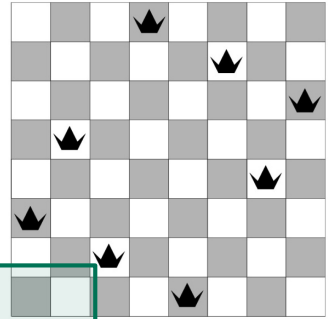
"au plus une reine par ligne"

on exprime d'abord le contraire

→ il existe au moins deux reines en ligne 1, et plus précisément

il existe $j, k \in \{1, \dots, 8\}$ tels que $j \neq k$

et $(1, j)$ contient une reine et $(1, k)$ aussi



✓
 $j, k \in \{1, \dots, 8\}$
 $j \neq k$
 $x_{1,j} \wedge x_{1,k}$

ON PREND
LA NEGATION

$$\neg \left(\bigvee_{\substack{j, k \in \{1, \dots, 8\} \\ j \neq k}} x_{1,j} \wedge x_{1,k} \right)$$

$$= \bigwedge_{\substack{j, k \in \{1, \dots, 8\} \\ j \neq k}} (\neg x_{1,j} \vee \neg x_{1,k})$$

LE PAIRE POUR TOUTES LES LIGNES:

$$\varphi_2 = \bigwedge_{i \in \{1, \dots, 8\}} \bigwedge_{\substack{j, k \in \{1, \dots, 8\} \\ j \neq k}} (\neg x_{i,j} \vee \neg x_{i,k})$$

$\varphi_1 \wedge \varphi_2$ exprime
que chaque
ligne a exactement
une reine

PAS D'ATTAQUE HORIZONTALE

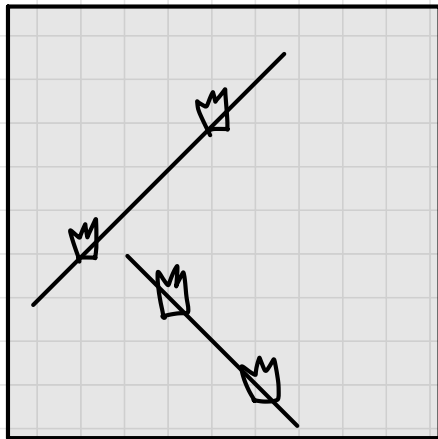
c'est φ_2 !

PAS D'ATTAQUE VERTICALE

$$\varphi_3 = \bigwedge_{i \in \{1, \dots, 8\}} \bigwedge_{\substack{j, k \in \{1, \dots, 8\} \\ j \neq k}} (\neg x_{j,i} \vee \neg x_{k,i})$$

PAS D'ATTAQUE DIAGONALE

une case (i, j) est
sur la même diagonale
qu'une case (k, l) ssi
 $|i - k| = |j - l|$



On ne veut pas

qu'il existe

(i, j) et (k, l)

① sur la même diagonale

② avec des reines.

$$\neg \left(\bigvee_{\substack{(i,j) \in \{1, \dots, 8\}^2 \\ (k,l) \in \{1, \dots, 8\}^2 \\ |i-k| = |j-l|}} x_{i,j} \wedge x_{k,l} \right) = \bigwedge_{\substack{(i,j) \in \{1, \dots, 8\}^2 \\ (k,l) \in \{1, \dots, 8\}^2 \\ |i-k| = |j-l|}} (\neg x_{i,j} \vee \neg x_{k,l})$$

$\uparrow$
 φ_4

ENSEMBLE FINAL DE CLAUSES

$\varphi = \varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ est déjà en FNC

→ l'ensemble des clauses est donc
donné par l'ensemble des disjonctions
de φ .

8 Reines - Codage en PySAT (1)

- ▶ codons le problème dans la librairie PySAT
- ▶ PySAT est une librairie Python qui fournit une interface permettant d'utiliser différents solveurs SAT.

8 Reines - Codage en PySAT (1)

```
from pysat.solvers import Minisat22
from pysat.solvers import Glucose4
from pysat.formula import CNF
from pysat.formula import IDPool

nb = 8 # nombre de reines

affichage_sol = True # affichage d'une solution

vpool = IDPool(start_from=1) # pour le stockage des identifiants entiers des couples (i,j)

cnf = CNF() # construction d'un objet formule en forme normale conjonctive (Conjunctive Normal Form)
```

- ▶ Pour PySAT, une variable booléenne est un entier positif, par exemple 7, et sa négation un entier négatif, par exemple, -7
- ▶ La classe IDPool permet de créer automatiquement un codage des objets qu'on veut manipuler, par exemple des paires d'entiers, par des entiers positifs
- ▶ cnf est un objet qui représente une formule en FNC. Chaque clause est une liste de littéraux, et une formule est une liste de clauses.

$$\bigwedge_{i \in \{1, \dots, 8\}} \left(\bigvee_{j \in \{1, \dots, 8\}} x_{i,j} \right)$$

8 Reines - Codage en PySAT (2)

Le code suivant ajoute un ensemble de clauses qui code la contrainte qu'il existe au moins une reine par ligne i :

```
print("Au moins un reine par ligne")

# au moins un reine par ligne
for i in range(nb):
    d = []
    for j in range(nb):
        d.append(vpool.id((i,j)))
    cnf.append(d)
```

- ▶ la variable d représente une disjonction de littéraux
- ▶ `vpool.id((i,j))` retourne un entier qui code la paire (i,j) , et l'ajoute à `vpool` si c'est la première fois que cette paire est rencontrée
- ▶ `cnf.append(d)` ajoute la clause d à l'ensemble des clauses

8 Reines - Codage en PySAT (3)

Le code suivant encode les contraintes qui expriment qu'il n'y a ni d'attaque en ligne ni d'attaque en colonne :

```
# pas d'attaque en ligne
print("Pas d'attaque en ligne")

for i in range(nb):
    for j in range(nb):
        for k in range(nb):
            if k > j:
                cnf.append([-vpool.id((i,j)), -vpool.id((i,k))])

# pas d'attaque en colonne
print("Pas d'attaque en colonne")

for i in range(nb):
    for j in range(nb):
        for k in range(nb):
            if k > j:
                cnf.append([-vpool.id((j,i)), -vpool.id((k,i))])
```

- `cnf.append([-vpool.id((i,j)), -vpool.id((i,k))])` ajoute une clause à deux éléments, en négation.

8 Reines - Codage en PySAT (4)

Ici on encode la contrainte “pas d'attaque diagonale” :

```
# pas d'attaque en diagonale

print("Pas d'attaque en diagonale")

# version lente
for i in range(nb):
    for j in range(nb):
        for k in range(nb):
            for l in range(nb):
                if abs(i-k)==abs(j-l) and i!=k:
                    cnf.append([-vpool.id((i,j)), -vpool.id((k,l))])
```

- Pour toute paire de cases (i,j) et (k,l) , le code `abs(i-k)==abs(j-l)` teste si elles sont sur la même diagonale
- ce code est lent quand `nb` est grand, on va voir une version optimisée.

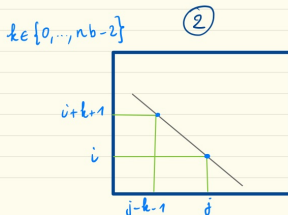
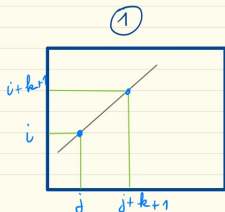
8 Reines - Codage en PySAT (5)

Version optimisée :

```
# version optimisee
for i in range(nb):
    for j in range(nb):
        for k in range(nb-1):
            if i+k+1<nb and j+k+1<nb:
                cnf.append([-vpool.id((i,j)), -vpool.id((i+k+1,j+k+1))])

for i in range(nb):
    for j in range(nb):
        for k in range(nb-1):
            if i+k+1<nb and j-k-1>=0:
                cnf.append([-vpool.id((i,j)), -vpool.id((i+k+1,j-k-1))])
```

Deux cas :



8 Reines - Codage en PySAT (6)

Lancement d'un solveur :

```
# phase de resolution

s = Minisat22(use_timer=True) # pour utiliser le solveur MiniSAT
# s = Glucose4(use_timer=True) # pour utiliser le solveur Glucose
s.append_formula(cnf.clauses, no_return=False)

print("Resolution...")
resultat = s.solve()
print("satisfaisable : " + str(resultat))

print("Temps de resolution : " + '{0:.2f}s'.format(s.time()))
```

- ▶ `s = Minisat22(use_timer=True)` crée un objet solveur MiniSAT (on pourrait utiliser un autre solveur)
- ▶ `use_timer=True` permet de compter le temps passé à la résolution
- ▶ `s.append_formula(cnf.clauses, no_return=False)` ajoute la formule au solveur
- ▶ `s.solve()` lance le solveur et retourne le résultat dans `resultat`
- ▶ `s.time()` retourne le temps passé à la résolution (depuis le dernier appel à la méthode `solve()`).

8 Reines - Codage en PySAT (7)

Remarque : La résolution est incrémentale : si on ajoute de nouvelles clauses à `s` et qu'on appelle une deuxième fois la méthode `solve()`, la résolution réutilise les calculs déjà faits lors du premier appel.

8 Reines - Codage en PySAT (8)

Affichage de la solution trouvée :

```
# affichage solution

if affichage_sol and resultat:

    print("\nVoici une solution: \n")

    for i in range(nb):
        for j in range(nb):
            if vpool.id((i,j)) in s.get_model():
                print("R ",end='')
            else:
                print("* ",end='')
        print("")
```

- ▶ `s.get_model()` retourne une interprétation qui satisfait la formule, sous forme de liste de littéraux.
- ▶ Par exemple, $V(1) = V(2) = 0$ et $V(3) = V(4) = V(5) = 1$ est représentée par la liste `[-1, -2, 3, 4, 5]`.

36 Problème du Sudoku (Au tableau)

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Comment choisir les propositions ?

Bien souvent, un problème de décision demande l'existence d'une relation binaire d'un ensemble fini A vers un ensemble fini B , qui satisfait certaines contraintes. Voici quelques exemples :

- ▶ **coloriage de graphes** : A est l'ensemble des sommets et B l'ensemble des couleurs.
- ▶ **n reines à placer sur un échiquier $n \times n$** : $A = \{1, \dots, n\}^2$ est l'ensemble des cases et $B = \{0, 1\}$.
- ▶ **Sudoku** : $A = \{1, \dots, 9\}^2$ est l'ensemble des cases et $B = \{1, \dots, 9\}$ l'ensemble des valeurs.

Comment choisir les propositions ?

Bien souvent, un problème de décision demande l'existence d'une relation binaire d'un ensemble fini A vers un ensemble fini B , qui satisfait certaines contraintes. Voici quelques exemples :

- ▶ **coloriage de graphes** : A est l'ensemble des sommets et B l'ensemble des couleurs.
- ▶ **n reines à placer sur un échiquier $n \times n$** : $A = \{1, \dots, n\}^2$ est l'ensemble des cases et $B = \{0, 1\}$.
- ▶ **Sudoku** : $A = \{1, \dots, 9\}^2$ est l'ensemble des cases et $B = \{1, \dots, 9\}$ l'ensemble des valeurs.

⇒ on peut alors tenter d'utiliser, pour tout $a \in A$ et $b \in B$, la proposition “ a est en relation avec b ”, notée $x_{a,b}$.

Que faire si on veut que la relation soit une fonction ?

- ▶ dans les trois exemples précédents, on voudrait que la relation soit une fonction totale de A vers B (exactement une couleur par sommet, exactement une valeur 0 ou 1 par case de l'échiquier, exactement une valeur par case pour le Sudoku)
- ▶ il faut donc exprimer les contraintes suivantes :
 - ▶ **au moins une** : $\bigwedge_{a \in A} (\bigvee_{b \in B} x_{a,b})$
 - ▶ **au plus une** : $\bigwedge_{a \in A, b, b' \in B, b \neq b'} \neg x_{a,b} \vee \neg x_{a,b'}$

39 Algorithme DPLL

- ▶ proposé par M. Davis, H. Putman, G. Logemann et D. Loveland en 1962.
- ▶ *principe de base* : engendrer et tester des solutions partielles
- ▶ Par exemple, pour tester la satisfiabilité de $(x \vee y \vee z) \wedge (x \vee \neg y \vee \neg z)$, au lieu d'essayer toutes les valuations possibles de x , y et z , on peut d'abord essayer les interprétations possibles de x .
- ▶ Ici, on se rend compte tout de suite que la valuation (partielle) $V(x) \mapsto 1$ satisfait la formule, peu importe l'interprétation des autres propositions.
- ▶ l'algorithme DPLL propose des critères pour choisir quelles variables tester en premier.

40 Interprétation Partielle

Définition

Une *interprétation partielle* est un assignement noté $x/1$ ou $x/0$, qui signifie que assigne la valeur 1 à x ou la valeur 0.

40 Interprétation Partielle

Définition

Une *interprétation partielle* est un assignement noté $x/1$ ou $x/0$, qui signifie que assigne la valeur 1 à x ou la valeur 0.

Lorsqu'on a choisi la valeur d'une proposition, on peut simplifier la formule. Par exemple, la formule $(x \vee y) \wedge (\neg x \vee z \vee \neg y)$ se simplifie en $z \vee \neg y$ sous l'interprétation partielle $x/1$.

Définissons de manière formelle cette notion.

41 Simplification sous une interprétation partielle

Etant donnée une clause C et une interprétation partielle x/b où $b \in \{0, 1\}$, la formule $C[x/b]$ est obtenue selon la règle suivante :

- ▶ Si C ne contient pas x ou $\neg x$, alors $C[x/b] = C$
- ▶ Sinon si C ne contient que x ou que $\neg x$, alors on considère les cas suivants :
 1. $C = x$ et $b = 1$, alors $C[x/b] = \top$
 2. $C = x$ et $b = 0$, alors $C[x/b] = \perp$
 3. $C = \neg x$ et $b = 0$, alors $C[x/b] = \top$
 4. $C = \neg x$ et $b = 1$, alors $C[x/b] = \perp$
- ▶ sinon si C contient x et $b = 1$ ou C contient $\neg x$ et $b = 0$, alors $C[x/b] = \top$
- ▶ sinon on retire de C les occurrences de x ou de $\neg x$

Si $\phi = \bigwedge_{i=1}^n C_i$ est une conjonction de clauses, alors $\phi[x/b] = \perp$ si il existe une clause C_i telle que $C_i[x/b] = \perp$, sinon $\phi[x/b] = \bigwedge_{i: C_i[x/b] \neq \top} C_i[x/b]$ (on simplifie toutes les clauses et on retire celle qui deviennent égales à $\top$). Si toutes les clauses se simplifient en $\top$, alors $\phi[x/b] = \top$.

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

► $a[x/1] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

► $a[x/1] = \top$,

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

► $a[x/1] = \top$, $b[x/1] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$,

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] = \neg y \vee \neg z$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] = \neg y \vee \neg z$
- ▶ $(a \wedge b)[x/0] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] = \neg y \vee \neg z$
- ▶ $(a \wedge b)[x/0] = (y \vee z) \wedge (\neg y \vee \neg z)$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] = \neg y \vee \neg z$
- ▶ $(a \wedge b)[x/0] = (y \vee z) \wedge (\neg y \vee \neg z)$
- ▶ $(a \wedge b)[x/0][y/0] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] = \neg y \vee \neg z$
- ▶ $(a \wedge b)[x/0] = (y \vee z) \wedge (\neg y \vee \neg z)$
- ▶ $(a \wedge b)[x/0][y/0] = z$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] = \neg y \vee \neg z$
- ▶ $(a \wedge b)[x/0] = (y \vee z) \wedge (\neg y \vee \neg z)$
- ▶ $(a \wedge b)[x/0][y/0] = z$
- ▶ $(a \wedge b)[x/0][y/0][z/1] =$

42 Remarque et exemples

Remarque

$\phi[x/b]$ est une formule, on peut donc lui réappliquer une autre interprétation partielle $[y/b']$ et obtenir une nouvelle formule $(\phi[x/b])[y/b']$ qu'on notera simplement $\phi[x/b][y/b']$.

Exemples

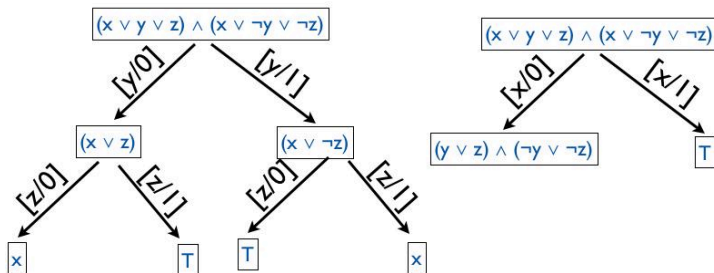
Soit $a = x \vee y \vee z$ et $b = x \vee \neg y \vee \neg z$. Alors :

- ▶ $a[x/1] = \top$, $b[x/1] = \top$, $(a \wedge b)[x/1] = \top$
- ▶ $b[y/1] = x \vee \neg z$
- ▶ $b[z/1] = x \vee \neg y$
- ▶ $a[x/0] = y \vee z$
- ▶ $b[x/0] = \neg y \vee \neg z$
- ▶ $(a \wedge b)[x/0] = (y \vee z) \wedge (\neg y \vee \neg z)$
- ▶ $(a \wedge b)[x/0][y/0] = z$
- ▶ $(a \wedge b)[x/0][y/0][z/1] = \top$

43 Proposition Pivot

- l'algorithme DPLL va essayer des interprétations partielles, la proposition choisie est la *proposition pivot*. Il va successivement essayer de mettre la proposition à vrai ou à faux, et tester récursivement la satisfaisabilité de formules simplifiées obtenues.
- le choix de la proposition choisie peut fortement influencer le résultat comme on peut le voir sur cet exemple :

$$\phi = (x \vee y \vee z) \wedge (x \vee \neg y \vee \neg z)$$



44 Premier critère de choix : clauses unitaires

- ▶ Une clause unitaire est une clause qui ne contient qu'un seul littéral x ou $\neg x$
- ▶ une clause unitaire force donc le choix de la valeur de la proposition. Par exemple dans $x \wedge (y \vee \neg z)$, il y a une clause unitaire x et on sait que pour satisfaire cette formule, il faut nécessairement interpréter x par 1
- ▶ l'algorithme DPLL choisi en priorité la proposition d'une clause unitaire comme proposition pivot.
- ▶ la formule simplifiée obtenue peut elle même contenir des clauses unitaires : *propagation de clauses unitaires*
- ▶ c'est une optimisation essentielle des solveurs SAT, qui passent l'essentiel de leur temps à faire de la propagation de clauses unitaires.

45 Propagation de clauses unitaires : exemple

Prenons

$$\phi = (x \vee y) \wedge \neg y \wedge (\neg x \vee y \vee \neg z)$$

Alors

$$\phi[y/0] = x \wedge (\neg x \vee \neg z)$$

ce qui donne la nouvelle clause unitaire x qui impose donc d'assigner la valeur 1 à x :

$$\phi[y/0][x/1] = \neg z$$

qui ne contient qu'une seule clause, unitaire, et on obtient finalement :

$$\phi[y/0][x/1][z/0] = \top$$

Donc ϕ est satisfaisable avec l'interprétation $V(x) = 1$ et $V(y) = V(z) = 0$.

46 Deuxième critère de choix : proposition à polarité unique

Prenons la formule

$$\phi = (x \vee \neg y \vee z) \wedge (x \vee \neg z) \wedge (y \vee z) \wedge (x \vee \neg y)$$

- ▶ dans cette formule, x apparaît toujours positivement, on peut donc directement lui assigner la valeur 1 sans être obligé de tester la valeur 0.
- ▶ on peut de même avoir un effet cascade : par exemple avec

$$(x \vee \neg y) \wedge (y \vee \neg z_1) \wedge (\neg z_1 \vee \neg z_2)$$

(Exercice)

47 Algorithme DPLL

Principe

- ▶ génération d'interprétation partielle
- ▶ la proposition pivot est choisie avec le critère de clause unitaire d'abord, et de polarité ensuite
- ▶ sinon, lorsque les deux critères ne s'appliquent pas, une proposition est choisie au hasard

48 Algorithme DPLL

Fonction $DPLL(\phi)$ = retourne VRAI ssi ϕ est satisfaisable

1. si $\phi = \top$ alors retourner VRAI (la formule est satisfaisable)
2. sinon si $\phi = \perp$ alors retourner FAUX (la formule n'est pas satisfaisable)
3. sinon si ϕ contient une clause unitaire x , alors retourner $DPLL(\phi[x/1])$
4. sinon si ϕ contient une clause unitaire $\neg x$, alors retourner $DPLL(\phi[x/0])$
5. sinon si ϕ contient une proposition x de polarité toujours positive, alors retourner $DPLL(\phi[x/1])$
6. sinon si ϕ contient une proposition x de polarité toujours négative, alors retourner $DPLL(\phi[x/0])$
7. sinon dans tous les autres cas, choisir une proposition x au hasard et retourner ($DPLL(\phi[x/0])$ ou $DPLL(\phi[x/1])$)

49 Algorithmes DPLL : Exemple

$$\begin{array}{l}
 (x_1 \vee \neg x_2 \vee y_1 \vee \neg y_2 \vee \neg z_2 \vee \neg z_4) \quad \neg \\
 \wedge (x_2 \vee y_1) \wedge (x_2 \vee y_1 \vee y_2 \vee z_1 \vee z_4) \wedge (x_2 \vee \neg y_2 \vee z_1 \vee \neg z_2) \\
 \wedge (x_2 \vee \neg y_1 \vee z_3 \vee \neg z_4) \wedge (x_2 \vee \neg y_2 \vee z_2 \vee \neg z_3) \\
 \wedge (\neg x_2 \vee \neg y_1) \wedge (\neg x_2 \vee \neg y_1 \vee \neg y_2) \wedge (\neg x_2 \vee y_1 \vee y_2) \wedge (\neg x_2 \vee \neg y_2 \vee z_1) \wedge (\neg x_2 \vee \neg z_1 \vee z_2) \\
 \wedge (\neg z_3 \vee \neg z_4) \wedge (z_3 \vee z_4) \wedge (\neg z_3 \vee z_4) \quad \neg
 \end{array}$$

(Au tableau)

50 Transformation de Tseitin

- ▶ parfois, le problème qu'on cherche à résoudre ne s'exprime pas facilement par une formule en FNC
- ▶ Transformation de Tseitin : ajouter des nouvelles variables et des équivalences

Exemple

Considérons $\phi = (x \wedge q) \vee \neg(y \vee r)$. Dans la transformation de Tseitin, on va remplacer $x \wedge q$ par une nouvelle variable x_1 , et $\neg(y \vee r)$ par une nouvelle variable x_2 . Nous considérons alors la formule

$$(x_1 \vee x_2) \wedge (x_1 \leftrightarrow x \wedge q) \wedge (x_2 \leftrightarrow \neg(y \vee r))$$

Il reste encore à mettre les deux formules $(x_1 \leftrightarrow x \wedge q)$ et $(x_2 \leftrightarrow \neg(y \vee r))$ sous FNC.

51 Transformation de Tseitin

Exemple (Suite)

On a la série d'équivalences suivantes :

$$\begin{aligned} & x_1 \leftrightarrow x \wedge q \\ \equiv \end{aligned}$$

51 Transformation de Tseitin

Exemple (Suite)

On a la série d'équivalences suivantes :

$$\begin{aligned} & x_1 \leftrightarrow x \wedge q \\ \equiv & (x_1 \rightarrow (x \wedge q)) \wedge ((x \wedge q) \rightarrow x_1) \\ \equiv & \end{aligned}$$

51 Transformation de Tseitin

Exemple (Suite)

On a la série d'équivalences suivantes :

$$\begin{aligned} & x_1 \leftrightarrow x \wedge q \\ \equiv & (x_1 \rightarrow (x \wedge q)) \wedge ((x \wedge q) \rightarrow x_1) \\ \equiv & (\neg x_1 \vee (x \wedge q)) \wedge (\neg(x \wedge q) \vee x_1) \\ \equiv & \end{aligned}$$

51 Transformation de Tseitin

Exemple (Suite)

On a la série d'équivalences suivantes :

$$\begin{aligned} & x_1 \leftrightarrow x \wedge q \\ \equiv & (x_1 \rightarrow (x \wedge q)) \wedge ((x \wedge q) \rightarrow x_1) \\ \equiv & (\neg x_1 \vee (x \wedge q)) \wedge (\neg(x \wedge q) \vee x_1) \\ \equiv & (\neg x_1 \vee x) \wedge (\neg x_1 \vee q) \wedge (\neg x \vee \neg q \vee x_1) \end{aligned}$$

et la série suivante :

$$\begin{aligned} & x_2 \leftrightarrow \neg(y \vee r) \\ \equiv & \end{aligned}$$

51 Transformation de Tseitin

Exemple (Suite)

On a la série d'équivalences suivantes :

$$\begin{aligned} & x_1 \leftrightarrow x \wedge q \\ \equiv & (x_1 \rightarrow (x \wedge q)) \wedge ((x \wedge q) \rightarrow x_1) \\ \equiv & (\neg x_1 \vee (x \wedge q)) \wedge (\neg(x \wedge q) \vee x_1) \\ \equiv & (\neg x_1 \vee x) \wedge (\neg x_1 \vee q) \wedge (\neg x \vee \neg q \vee x_1) \end{aligned}$$

et la série suivante :

$$\begin{aligned} & x_2 \leftrightarrow \neg(y \vee r) \\ \equiv & (x_2 \rightarrow \neg(y \vee r)) \wedge (\neg(y \vee r) \rightarrow x_2) \\ \equiv & \end{aligned}$$

51 Transformation de Tseitin

Exemple (Suite)

On a la série d'équivalences suivantes :

$$\begin{aligned}x_1 &\leftrightarrow x \wedge q \\ \equiv & (x_1 \rightarrow (x \wedge q)) \wedge ((x \wedge q) \rightarrow x_1) \\ \equiv & (\neg x_1 \vee (x \wedge q)) \wedge (\neg(x \wedge q) \vee x_1) \\ \equiv & (\neg x_1 \vee x) \wedge (\neg x_1 \vee q) \wedge (\neg x \vee \neg q \vee x_1)\end{aligned}$$

et la série suivante :

$$\begin{aligned}x_2 &\leftrightarrow \neg(y \vee r) \\ \equiv & (x_2 \rightarrow \neg(y \vee r)) \wedge (\neg(y \vee r) \rightarrow x_2) \\ \equiv & (\neg x_2 \vee \neg y \wedge \neg r) \wedge (y \vee r \vee x_2) \\ \equiv & \end{aligned}$$

51 Transformation de Tseitin

Exemple (Suite)

On a la série d'équivalences suivantes :

$$\begin{aligned}x_1 &\leftrightarrow x \wedge q \\ \equiv & (x_1 \rightarrow (x \wedge q)) \wedge ((x \wedge q) \rightarrow x_1) \\ \equiv & (\neg x_1 \vee (x \wedge q)) \wedge (\neg(x \wedge q) \vee x_1) \\ \equiv & (\neg x_1 \vee x) \wedge (\neg x_1 \vee q) \wedge (\neg x \vee \neg q \vee x_1)\end{aligned}$$

et la série suivante :

$$\begin{aligned}x_2 &\leftrightarrow \neg(y \vee r) \\ \equiv & (x_2 \rightarrow \neg(y \vee r)) \wedge (\neg(y \vee r) \rightarrow x_2) \\ \equiv & (\neg x_2 \vee \neg y \wedge \neg r) \wedge (y \vee r \vee x_2) \\ \equiv & (\neg x_2 \vee \neg y) \wedge (\neg x_2 \vee \neg r) \wedge (y \vee r \vee x_2)\end{aligned}$$

52 Transformation de Tseitin

Exemple (Suite)

La formule ϕ de départ était $(x \wedge q) \vee \neg(y \vee r)$ qu'on a transformé en :

$$(x_1 \vee x_2) \wedge (x_1 \leftrightarrow x \wedge q) \wedge (x_2 \leftrightarrow (y \vee r))$$

On a montré les équivalences suivantes :

$$\begin{aligned} x_1 \leftrightarrow x \wedge y &\equiv (\neg x_1 \vee p) \wedge (\neg x_1 \vee q) \wedge (\neg x \vee \neg y \vee x_1) \\ x_2 \leftrightarrow (\neg x_2 \vee \neg q) \wedge (\neg x_2 \vee \neg r) \wedge (y \vee r \vee x_1) \end{aligned}$$

Donc finalement on obtient la formule sous FNC suivante $\psi =$

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee p) \wedge (\neg x_1 \vee q) \wedge (\neg x \vee \neg y \vee x_1) \wedge (\neg x_2 \vee \neg q) \wedge (\neg x_2 \vee \neg r) \wedge (y \vee r \vee x_1)$$

Note : la satisfaisabilité est préservée, i.e. ϕ est satisfaisable ssi ψ est satisfaisable.

53 Transformation de Tseitin

- la technique de Tseitin sera particulièrement intéressante lorsqu'on devra mettre sous FNC des formules qui sont sous forme normale *disjonctive*

$$C_1 \vee C_2 \vee \dots \vee C_n$$

où les C_i sont des conjonctions de littéraux.

- on va introduire une variable x_i pour chaque C_i , et on obtient la formule

$$(x_1 \vee x_2 \vee \dots \vee x_n) \wedge (x_1 \leftrightarrow C_1) \wedge \dots \wedge (x_n \leftrightarrow C_n)$$

- supposons que $C_i = l_1 \wedge \dots \wedge l_k$ où les l_j sont des littéraux, alors mettre $x_i \leftrightarrow C_i$ sous FNC est assez simple :

$$x_i \leftrightarrow C_i \equiv \left(\bigwedge_{j=1}^k (\neg x_i \vee l_j) \right) \wedge \left(x_i \vee \bigvee_{j=1}^k \neg l_j \right)$$

qui est bien sous FNC.

54 La contrainte “Exactement Une”

Supposons qu'on ait n variables $x_0, \dots, x_{n-1}$ et qu'on veuille exprimer qu'exactement une de ces variables doit être vraie.

Solution Naïve

On considère les deux formules suivantes (en conjonction)

- ▶ **Au moins une** : $\bigvee_{i=0}^{n-1} x_i$
- ▶ **Au plus une** : $\bigwedge_{0 \leq i < j < n} \neg x_i \vee \neg x_j$

Il y a donc $n(n-1)/2 + 1$ clauses.

55 La contrainte “Exactement Une” : Encodage Binaire

Supposons que $n = 2^k$. On utilise k nouvelles variables $b_1, \dots, b_k$, et pour tout $i \in \{0, \dots, n-1\}$, et notons $\bar{i} = \overline{\alpha_1 \dots \alpha_k}$ son écriture en base 2. On note :

$$B_i = \ell_1 \wedge \dots \wedge \ell_k \text{ où } \forall 1 \leq j \leq k, \ell_j = \begin{cases} b_j & \text{si } \alpha_j = 1 \\ \neg b_j & \text{sinon.} \end{cases}$$

Par exemple pour $n = 8$, on a :

- ▶ $B_0 = \neg b_1 \wedge \neg b_2 \wedge \neg b_3$
- ▶ $B_1 = \neg b_1 \wedge \neg b_2 \wedge b_3$
- ▶ $B_7 = b_1 \wedge b_2 \wedge b_3$
- ▶ **Remarque** : si $V \models B_i$, alors $V \not\models B_j$ pour tout $j \neq i$

55 La contrainte “Exactement Une” : Encodage Binaire

Supposons que $n = 2^k$. On utilise k nouvelles variables $b_1, \dots, b_k$, et pour tout $i \in \{0, \dots, n-1\}$, et notons $\bar{i} = \overline{\alpha_1 \dots \alpha_k}$ son écriture en base 2. On note :

$$B_i = \ell_1 \wedge \dots \wedge \ell_k \text{ où } \forall 1 \leq j \leq k, \ell_j = \begin{cases} b_j & \text{si } \alpha_j = 1 \\ \neg b_j & \text{sinon.} \end{cases}$$

Par exemple pour $n = 8$, on a :

- ▶ $B_0 = \neg b_1 \wedge \neg b_2 \wedge \neg b_3$
- ▶ $B_1 = \neg b_1 \wedge \neg b_2 \wedge b_3$
- ▶ $B_7 = b_1 \wedge b_2 \wedge b_3$
- ▶ **Remarque** : si $V \models B_i$, alors $V \not\models B_j$ pour tout $j \neq i$

Encodage Binaire

- ▶ **Au moins une** : $\bigvee_{i=0}^{n-1} x_i$
- ▶ **Au plus une** : $\bigwedge_{0 \leq i < n} x_i \rightarrow B_i \equiv \bigwedge_{0 \leq i < n} \bigwedge_{\ell \in B_i} (\neg x_i \vee \ell)$

Il y a donc $n.k + 1 = n.\log_2(n) + 1$ clauses.

56 Exemple pour $n = 8$ et la contrainte “au plus une”

On obtient l'ensemble de clauses S_8 suivant :

$$\begin{array}{cccccccc}
 \neg x_0 \vee \neg b_1 & \neg x_1 \vee \neg b_1 & \neg x_2 \vee \neg b_1 & \neg x_3 \vee \neg b_1 & \neg x_4 \vee b_1 & \neg x_5 \vee b_1 & \neg x_6 \vee b_1 & \neg x_7 \vee b_1 \\
 \neg x_0 \vee \neg b_2 & \neg x_1 \vee \neg b_2 & \neg x_2 \vee b_2 & \neg x_3 \vee b_2 & \neg x_4 \vee \neg b_2 & \neg x_5 \vee \neg b_2 & \neg x_6 \vee b_2 & \neg x_7 \vee b_2 \\
 \neg x_0 \vee \neg b_3 & \neg x_1 \vee b_3 & \neg x_2 \vee \neg b_3 & \neg x_3 \vee b_3 & \neg x_4 \vee \neg b_3 & \neg x_5 \vee b_3 & \neg x_6 \vee \neg b_3 & \neg x_7 \vee b_3
 \end{array}$$

Correction de l'encodage Soit V tel que $V \models S_8$, alors si $V(x_i) = 1$, on a $V \models B_i$, par définition de l'encodage binaire. S'il existe une autre variable x_j telle que $V(x_j) = 1$, alors $V \models B_j$, ce qui contredit la remarque précédente.

57 A retenir

Principe (souvent) valable pour les SAT solveurs

Ne pas hésiter à introduire des nouvelles variables pour minimiser le nombre de clauses !

58 Variantes de SAT

- ▶ 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
 - ▶ Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
 - ▶ 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).

58 Variantes de SAT

- ▶ 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
 - ▶ Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
 - ▶ 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- ▶ QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \dots \forall q_n \exists q_n. \phi$$

où ϕ est une formule en FNC construite sur les propositions $x_1, q_1, \dots, x_n, q_n$.

58 Variantes de SAT

- ▶ 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
 - ▶ Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
 - ▶ 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- ▶ QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \dots \forall q_n \exists q_n. \phi$$

où ϕ est une formule en FNC construite sur les propositions $x_1, q_1, \dots, x_n, q_n$.

- ▶ par exemple : $\forall x \exists q. (x \vee q) \wedge (\neg x \vee \neg q)$.

58 Variantes de SAT

- ▶ 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
 - ▶ Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
 - ▶ 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- ▶ QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \dots \forall q_n \exists q_n. \phi$$

où ϕ est une formule en FNC construite sur les propositions $x_1, q_1, \dots, x_n, q_n$.

- ▶ par exemple : $\forall x \exists q. (x \vee q) \wedge (\neg x \vee \neg q)$.
- ▶ Si p est vraie, alors q doit être fausse, et si p est fausse, alors q doit être vraie.

58 Variantes de SAT

- ▶ 2-SAT : les clauses ne contiennent qu'au plus deux littéraux.
 - ▶ Par exemple $(x_1 \vee \neg x_2) \wedge x_3 \wedge (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2)$
 - ▶ 2-SAT est solvable en temps polynomial (alors que 3-SAT est NP-complet).
- ▶ QSAT : décider la satisfaisabilité de formules de la forme

$$\forall x_1 \exists q_1 \forall x_2 \exists q_2 \dots \forall q_n \exists q_n. \phi$$

où ϕ est une formule en FNC construite sur les propositions

$x_1, q_1, \dots, x_n, q_n$.

- ▶ par exemple : $\forall x \exists q. (x \vee q) \wedge (\neg x \vee \neg q)$.
- ▶ Si p est vraie, alors q doit être fausse, et si p est fausse, alors q doit être vraie.
- ▶ QSAT est PSPACE-complet (QSAT peut être résolu en espace polynomial et tous les problèmes pouvant être résolus en espace polynomial peuvent se réduire au problème QSAT en temps polynomial, autrement dit, QSAT est aussi difficile que tous les problèmes pouvant être résolu en espace polynomial)

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ on peut satisfaire au plus deux clauses.

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ on peut satisfaire au plus deux clauses.
 - ▶ Pourquoi MAX-SAT est aussi difficile que SAT ?

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ on peut satisfaire au plus deux clauses.
 - ▶ Pourquoi MAX-SAT est aussi difficile que SAT ?
- ▶ WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r , et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ on peut satisfaire au plus deux clauses.
 - ▶ Pourquoi MAX-SAT est aussi difficile que SAT ?
- ▶ WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r , et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $\text{poids}(x) = 1$, $\text{poids}(y) = 1$ et $\text{poids}(\neg x \vee \neg y) = 2$ et $r = 3$

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ on peut satisfaire au plus deux clauses.
 - ▶ Pourquoi MAX-SAT est aussi difficile que SAT ?
- ▶ WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r , et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $\text{poids}(x) = 1$, $\text{poids}(y) = 1$ et $\text{poids}(\neg x \vee \neg y) = 2$ et $r = 3$ En mettant x et y à vrai, on ne satisfait pas la dernière clause, et on obtient le poids total $1 + 1 = 2$. En mettant x à vrai et y à faux, on satisfait la première et la dernière clause, et on obtient le poids total $1 + 2 = 3$.

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ on peut satisfaire au plus deux clauses.
 - ▶ Pourquoi MAX-SAT est aussi difficile que SAT ?
- ▶ WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r , et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $\text{poids}(x) = 1$, $\text{poids}(y) = 1$ et $\text{poids}(\neg x \vee \neg y) = 2$ et $r = 3$ En mettant x et y à vrai, on ne satisfait pas la dernière clause, et on obtient le poids total $1 + 1 = 2$. En mettant x à vrai et y à faux, on satisfait la première et la dernière clause, et on obtient le poids total $1 + 2 = 3$.
 - ▶ Pourquoi WEIGHTED-MAX-SAT est aussi difficile que MAX-SAT ?

59 Variantes de SAT

- ▶ MAX-SAT : étant donnée une formule ϕ en FNC et un entier $k \in \mathbb{N}$, peut-on satisfaire au moins k clauses ?
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ on peut satisfaire au plus deux clauses.
 - ▶ Pourquoi MAX-SAT est aussi difficile que SAT ?
- ▶ WEIGHTED-MAX-SAT : on attribue des poids à chaque clause, on se donne un entier r , et on veut savoir s'il on peut satisfaire un ensemble de clauses dont la somme des poids est au moins r
 - ▶ Par exemple $x \wedge y \wedge (\neg x \vee \neg y)$ avec $\text{poids}(x) = 1$, $\text{poids}(y) = 1$ et $\text{poids}(\neg x \vee \neg y) = 2$ et $r = 3$ En mettant x et y à vrai, on ne satisfait pas la dernière clause, et on obtient le poids total $1 + 1 = 2$. En mettant x à vrai et y à faux, on satisfait la première et la dernière clause, et on obtient le poids total $1 + 2 = 3$.
 - ▶ Pourquoi WEIGHTED-MAX-SAT est aussi difficile que MAX-SAT ?
MAX-SAT est le cas particulier de WEIGHTED-MAX-SAT où le poids de chaque clause vaut 1.

60 Références

- ▶ http://en.wikipedia.org/wiki/Boolean_satisfiability_problem
- ▶ cours/tutoriaux : <http://www.cse.unsw.edu.au/~tw/sat/>
- ▶ <http://www.pps.jussieu.fr/~treinen/teaching/> (cours de Ralf Treinen)
- ▶ <http://www.cs.wm.edu/~idillig/cs780-02/lecture3.pdf>
- ▶ Handbook of satisfiability. A. Biere, M. Heule, H. van Maaren, T. Walsh. IOS Press. 2008. 981 pages.